API Security
Risk Analysis Report

What Happens When You Just... Ask An API For Data

APIs tested: JSONPlaceholder (jsonplaceholder.typicode.com) and ReqRes (reqres.in)
Programme: Future Interns - Cyber Security Task 3 (2026)
Prepared by: Amen Zelalem Ayalew
Date: 10 October 2026

1. The Short Version

Both of these APIs are public demo APIs, built on purpose for people to practice against, so nothing here was actually broken into. What I did instead was send normal, read-only requests and pay close attention to what came back: was a login needed, how much data showed up, did bad input get handled cleanly, and was there any limit on how many times I could ask.

JSONPlaceholder turned out to have five real issues worth flagging, all stemming from one root cause: there is no authentication at all, anywhere, on any endpoint. ReqRes, by contrast, now requires an API key on its main endpoints, so I included it as a short, clean comparison of what the fix actually looks like in practice.

If this were a real SaaS product built the same way as JSONPlaceholder: any outsider could script a loop, hit every user ID in order, and walk away with a full export of every customer's name, email, phone number and home address, with nothing to stop them and nothing logging that it happened.

High	Medium	Low	Total
1	4	1	6

Fix first: Put a real authentication check in front of every endpoint that returns personal data. Everything else on this list gets easier to justify fixing once that one is done.

2. How I Did This

APIs tested: JSONPlaceholder, a free fake-data API meant for practice, and ReqRes, a similar public testing API, both listed in the task itself as safe to use.
What I actually did: sent plain GET requests to several endpoints, read the raw JSON that came back, checked the response headers for things like rate-limit information, and tried one deliberately invalid ID to see how errors were handled. Nothing was flooded, nothing was brute-forced, and no write requests (POST, PUT, DELETE) were sent against real data.
Endpoints checked: /users/1, /users (list), /users/11 (does not exist), /posts/1, /posts/1/comments, and ReqRes's /api/users for comparison.

Tool	What I used it for
Direct HTTP GET requests	Reading real responses from each endpoint, exactly as Postman or a browser would sh
Response inspection	Checking which fields came back and whether they were more than a normal app would
Deliberate bad input (invalid ID)	Seeing whether the API fails cleanly or leaks information through its error behaviour
Header inspection	Looking for authentication requirements and rate-limit signals

How I rated each one: **High** means real customer data is exposed with nothing blocking it. **Medium** means it makes an attack easier or leaks more than it should. **Low** means good practice worth noting, not a problem.

3. Findings At a Glance

ID	Finding	Risk
F-01	No authentication needed to read full user records	High
F-02	Every field gets sent back, even ones a client app would never need	Medium
F-03	Asking for a user that doesn't exist still comes back 'OK'	Medium
F-04	No sign of any request limit, even after repeated calls	Medium
F-05	Comments expose real-looking email addresses with no login	Medium
F-06	ReqRes does this one correctly, worth noting as the good example	Low

4. Going Through Each Finding

F-01 No authentication needed to read full user records — High

Request I sent

```
GET https://jsonplaceholder.typicode.com/users/1
```

What came back


```
{ "id": 1, "name": "Leanne Graham", "username": "Bret", "email": "Sincere@april.biz", "address":
{ "street": "Kulas Light", "suite": "Apt. 556", "city": "Gwenborough", "zipcode": "92998-3874",
"geo": { "lat": "-37.3159", "lng": "81.1496" } }, "phone": "1-770-736-8031 x56442", "website":
"hildegard.org", "company": { "name": "Romaguera-Crona", ... } }
```


What I found

I sent a plain GET request to the users endpoint with no API key, no token, no login of any kind, and it handed back the full profile straight away: full name, email, phone number, home address, and even GPS-style coordinates.

Why it matters for a business

If a real SaaS product built its user API the same way, anyone on the internet could pull every customer's contact details just by guessing or looping through ID numbers. No password needed, no logging in, nothing.

What to do about it

Put every endpoint that returns personal data behind a real authentication check (an API key or a login token), and only return the fields the calling app actually needs.

F-02 Every field gets sent back, even ones a client app would never need — Medium

Request I sent

```
GET https://jsonplaceholder.typicode.com/users/1
```

What came back

```
Full address block, GPS coordinates (lat/lng), phone extension, and internal-sounding company
'catchPhrase' and 'bs' fields are all included in a single user lookup.
```

What I found

A normal app screen showing 'your profile' rarely needs someone's GPS coordinates or an internal company tagline. This endpoint sends all of it anyway, every time, to every caller.

Why it matters for a business

This is what's called excessive data exposure. Even if the data were locked behind a login, sending extra fields means more gets exposed the moment something goes wrong, like a leaked token or a buggy frontend that logs responses.

What to do about it

Design each endpoint to return only the fields that specific screen or feature needs, not the entire database row.

F-03 Asking for a user that doesn't exist still comes back 'OK' — Medium

Request I sent

```
GET https://jsonplaceholder.typicode.com/users/11 (only 10 users exist)
```

What came back

```
HTTP 200 OK, with an empty object {} in the response body, instead of a 404 Not Found.
```

What I found

I asked for a user ID that's one past the last real one. A well-behaved API says '404, not found.' This one says '200, here you go' and just hands back nothing.

Why it matters for a business

This sounds small, but it means someone probing the API can't easily tell the difference between 'this ID doesn't exist' and 'this ID exists but you're not allowed to see it.' That blurry line makes it easier to map out a system from the outside.

What to do about it

Return a proper 404 status for IDs that don't exist, and a proper 403 for IDs that exist but the caller isn't allowed to see.

F-04 No sign of any request limit, even after repeated calls — Medium

Request I sent

```
Multiple GET requests sent back to back to /users/1, /posts/1, /posts/1/comments
```

What came back

```
No X-RateLimit-Limit, X-RateLimit-Remaining, or Retry-After style headers showed up in any
response, and nothing slowed down or got blocked.
```

What I found

Real APIs usually tell you, in the headers, how many requests you have left before they cut you off. I didn't see any of that here, on any endpoint I checked.

Why it matters for a business

Without a limit, nothing stops someone from writing a simple script that loops through every possible ID and pulls the entire user table in a few seconds. That turns a 'one record at a time' API into a full data export tool.

What to do about it

Add rate limiting per API key or per IP address, and return clear headers showing the caller how close they are to the limit.

F-05 Comments expose real-looking email addresses with no login — Medium

Request I sent

```
GET https://jsonplaceholder.typicode.com/posts/1/comments
```

What came back

```
[ { "postId": 1, "id": 1, "name": "id labore ex et quam laborum", "email": "Eliseo@gardner.biz",
"body": "..." }, { "postId": 1, "id": 2, ... "email": "Jayne_Kuhic@sydney.com", ... }, ... 5
comments returned, each with a full email address ... ]
```

What I found

Pulling comments on a single post also hands back a real-shaped email address for every commenter, again with nothing checking who's asking.

Why it matters for a business

Comment and feedback systems are often treated as 'low risk' by teams building an API, but email addresses are still personal data. If this were a real support or forum product, this single endpoint alone would leak every commenter's email to anyone who found the URL.

What to do about it

Strip or mask personal fields like email from any response visible to the public, and only show full contact details to someone authenticated to see them.

F-06 ReqRes does this one correctly, worth noting as the good example — Low

Request I sent

```
GET https://reqres.in/api/users?page=2
```

What came back

```
As of ReqRes's current setup, classic endpoints like /api/users now require a valid x-api-key
header. A request with no key, or an empty one, is rejected with 401 Unauthorized.
```

What I found

I checked ReqRes as a second example, since it's listed in the task alongside JSONPlaceholder. Unlike JSONPlaceholder, it has since added a required API key for its main endpoints.

Why it matters for a business

This isn't a finding against the business, it's the opposite: it's what good practice looks like. I'm including it so the report shows both a working example and a broken one, side by side.

What to do about it

No fix needed here. This is the pattern F-01 should be copying.

5. Fixing This, In Order

When	What to do	Fixes
Now	Require a real API key or login token on every endpoint that returns personal data	F-01
Now	Return a proper 404 for IDs that don't exist, and a 403 for ones the caller can't access	F-03
Soon	Add rate limiting and return clear limit headers on every response	F-04
Soon	Trim each response down to only the fields that screen or feature actually needs	F-02, F-05
Ongoing	Review new endpoints against this same checklist before they ship	All

A note on scope: everything above came from plain GET requests and one deliberately invalid ID. I never tried to bypass a login, flood an endpoint, or touch anything outside the two public demo APIs named in the task. The findings are about what the API freely hands over by default, not about breaking in anywhere.

Reference: OWASP API Security Top 10, particularly API1 (Broken Object Level Authorization), API3 (Broken Object Property Level Authorization / excessive data exposure), and API4 (Unrestricted Resource Consumption).